

Project Report — Work Smarter, Not Harder

Course: Writing Software for Machine Learning (00950219), Technion — Spring 2026 **Team:** Shiri Haboob · Lior Ben Sidi · Elad Nachalieli **Repository:** LiorBenSidi/work-smarter-not-harder · **Due:** 23 Aug 2026

This is the living project report — the 23.08 deliverable required by the guidelines: an explanation of the app, a **feature × tests matrix drawn from the real suite**, and a **risk assessment**. It is kept in-repo and updated as the build progresses. Spec sources: the submitted `PROPOSAL.md` (graded 100/100 — the contract) and the rubric in `GUIDELINES.md` (75 build · +5 Job Queue · +10 Forum · +10 Azure deploy + CI/CD; supersedes `FEEDBACK.md`). Architecture detail lives in `DESIGN.md`; the phased plan in `ROADMAP.md`.

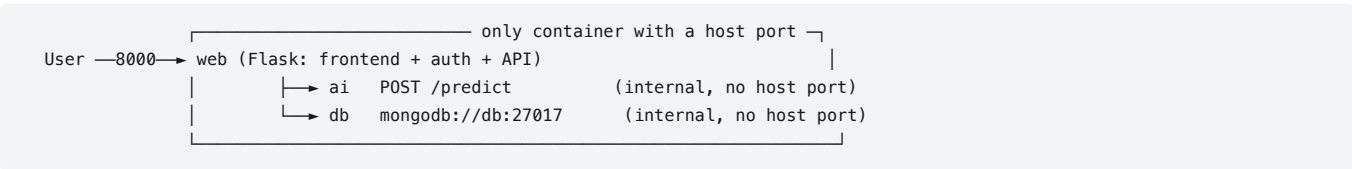
Last updated: 2026-07-11 · **Suite at this snapshot:** **764 tests, 728 passing / 36 environment-gated** (`pytest --collect-only`; the env-gated ones run in CI's `compose-e2e` job against the live containers). Since the 07-10 snapshot: +9 frontend design-pass guards, the AUTH-H1/H2 timing-oracle tests, and two browser E2E scenarios (forum avatars, profile grouping) that run on desktop **and** mobile in CI.

⚠ **Sections still owned by their planes.** §5 (risk), §2's deploy/scale/queue rows and §3's Elad rows are current as of this date. §1's API surface + data model now list the DM / SSE / notification / comment-vote endpoints and the `messages` / `notifications` collections (**Lior**), plus the media endpoints and the `media` collection (**Elad**, #160). Still open: §2's AI rows (F3 model, F5, F6/F7 engine, F4 live value) plus §3's `test_ai` scaffold row await the Random Forest (**Shiri**). The rubric is `GUIDELINES.md` (which superseded `FEEDBACK.md` on 8 Jul and added the **+5 Job Queue**).

1. What the application is

Work Smarter, Not Harder is an AI sports-coaching platform. Instead of only answering “*am I ready to train?*”, it answers *what to do next, what to improve first, how many calories to eat, whether the program is balanced, and whether to push, maintain, recover, or deload*. It combines a readiness classifier, a calorie recommendation, a workout/action-plan engine, a persistent history, and a community forum.

Architecture — three containers, only `web` exposed



- **web** — Flask app: server-rendered SPA frontend, authentication (werkzeug password hashing, CSRF double-submit), the JSON API, and orchestration of `ai` + `db`. Publishes host **8000** → **container 5000** (never 5000 on the host — macOS AirPlay squats there). The only user-facing container.
- **db** — stock `mongo:7`. Internal only; named volume for persistence; healthcheck + `restart`.
- **ai** — model inference + recommendation generation + the calorie engine. Internal only (`expose`, no host port).

The seams between containers are the only fixed shared contracts: the `web` → `ai` `POST /predict` request/response shape, and the `web` → `db` data API (`web/services/db.py`). Behind its contract each plane is implemented independently.

API surface (implemented)

Endpoint	Method	Feature
/register · /login · /logout · /me · /auth/config	POST/GET	F1 accounts, session, credential hints
/profile	GET · POST	F2 athlete profile
/checkin	POST	F3 readiness check-in → ai /predict → persist
/dashboard	GET	F8 dashboard (readiness + recommendations + calories)
/history	GET	F9 analysis history
/forum/posts	GET · POST	Forum: list / create
/forum/posts/<id>	GET · PATCH · DELETE	Forum: read / edit-own / delete-own (403 otherwise)
/forum/posts/<id>/comments	POST	Forum: comment
/forum/posts/<id>/vote	POST	Forum: post vote (strict +1 / -1)
/forum/posts/<id>/comments/<cid>/vote	POST	Forum: comment vote (strict +1 / -1)
/me/engagement	GET	Forum: the caller's received-vote totals (§3.3 "personal area" metric; counts only, never voter identities) — Elad's lane
/messages · /conversations · /conversations/<peer> · /users/search	POST · GET	Direct messages: send / list threads / read a thread / recipient search
/notifications · /notifications/read	GET · POST	Notifications feed (poll cursor) + mark-read
/events	GET	Real-time push — SSE text/event-stream (new posts, DMs, notifications)
/media · /media/<id> · /forum/posts/<id>/attachments · /messages/<peer>/attachments	POST · GET	Media upload / serve + bind-to-post / bind-to-DM (owner/participant-checked) — Elad's lane (#160)
/ · /health · /ready	GET	SPA entry · liveness probe · readiness (dependency) gate
/manifest.webmanifest · /sw.js	GET	PWA install manifest · service worker
ai:/predict	POST	readiness class + probabilities + recommendations (internal; web also reads an optional calorie target, added once the model lands)

Data model (MongoDB)

users { username (*the unique internal handle*), display_name (*non-unique*), email, password_hash } · profiles { username, age, gender, height, weight, goal, training_frequency } · analysis_history { username, metrics, assessment, calories, timestamp } · forum_posts { id, author, title, body, anonymous, score, votes, comments } · messages { id, sender, recipient, body, created_at, read } · notifications { id, user, type, text, actor, read, created_at } · media { id, owner, mime, size, target_type, target_id, peers, created_at } (*Elad, #160*).

2. Current build status (honest snapshot)

Legend:  done & tested ·  partial / in progress ·  not started. "Plane" is the owning container/aspect, not a per-line credit — see §6.

Feature / component	Status	Plane
F1 – User accounts (register / login / logout, werkzeug hashing, session)	✓	web
F2 – Athlete profile (validated, injection-safe)	✓	web
F3 – Readiness analysis – web path (check-in → <code>ai /predict</code> → persist → surface)	✓	web
F3 – Readiness analysis – the model (Random Forest, real /predict)	■	ai
F4 – Calorie recommendation (Mifflin–St Jeor, <code>ai/calories.py</code>)	● engine + 14 unit tests done; live value wires into <code>/predict</code> with the model	ai
F5 – Workout generator	■	ai
F6 – Program-balance analysis (folds into F7, pending Noam)	■	ai
F7 – Action plan / recommendations – web surfaces the list; engine	●	web ✓ · ai ■
F8 – Dashboard	✓	web
F9 – History tracking	✓	web
Online Forum – CRUD/UI (posts, comments, votes, anonymity, edit/delete-own)	✓	web
Online Forum – real-time layer (SSE push, DM, vote + DM notifications, media attachments)	✓	web (DM/SSE/notifications) · deploy (media, size caps)
Online Forum – received-engagement metric (§3.3 per-user vote total: <code>/me/engagement</code> + a Profile-screen card)	✓	deploy (Elad)
Online Forum – cold-seed content	● (idempotent seed script; content)	web ✓ · ai (content)
Data layer – <code>db.py</code> CRUD + indexes + <code>jsonSchema</code> validators + auth config + seed + backup	✓	web (data)
Week-9 observability – named loggers, console + rotating-file handlers, per-request access log	✓	web
Containers – 3-container <code>docker–compose</code> , only web exposed, healthchecks	✓	web
Fault tolerance – restart policies, <code>start_period</code> , web boots even if <code>ai</code> is down; degrade-not-crash	✓	web
CI gate – ruff → bandit → pytest (+ <code>mongo:7</code> service), PR-only branch-protected <code>main</code>	✓	web (CI)
Rate-limiting / anti-spam (flask-limiter) + upload size caps	✓	deploy/scale
Stress / load (locust + a dependency-free burst)	✓	deploy/scale
Cross-container test-runner (<code>docker–compose.test.yml</code> , CI job <code>compose–e2e</code>)	✓	deploy
Fault isolation – stop <code>ai</code> / stop <code>db</code> , web survives	✓	deploy
AI job queue (§2, +5) – bounded queue + <code>ProcessPoolExecutor</code> in front of the model	✓	ai (queue)
Azure VM deploy + CI/CD auto-deploy	✓ live (<code>app.worksmarternotharder.dev</code> ; auto-deploy on green <code>main</code> , /ready gate, auto-rollback)	web (pipeline) · deploy (live)
Parallel programming – measured CPU-bound path + horizontal <code>ai</code> scaling	✓ measured: pool 1→4 = 2.86× , <code>--scale ai=2</code> = 1.60× (<code>SCALING_REPORT.md</code>)	ai · deploy

In one sentence: the entire web tier, the whole data layer, observability, the 3-container build, and the CI gate are complete and **proven end-to-end** (a real `web` → `ai` → `db` request path, the real-Mongo integration suite, and in-container logging all verified live); the calorie engine is built + unit-tested but its value goes live once the model wires it into `/predict`, and the Random Forest model and the deploy / real-time / scale plane are the remaining build.

3. Feature × tests matrix (from the real suite)

Every cell names the **actual test file(s)** and count. This supersedes the aspirational Yes/No matrix in `PROPOSAL.md §13`. The rows below map each graded feature to representative test file(s); the totals under the table are the **full** suite by type (`pytest --collect-only`).

Feature / component	Unit	Integration	System	Stress	Security
F1 Accounts / Auth	test_auth_validation (24)	test_auth_flow (15)	test_e2e leg	—	test_auth (11) · test_csrf (6)
F2 Profile	test_profile_validation (27)	test_profile_flow (4)	test_e2e leg	—	test_profile (4)
F3 Readiness (web path)	test_checkin_validation (25) · test_ai_client (3)	test_checkin_flow (5) · test_web_ai (2)	test_e2e leg	■ (planned — locust)	—
F3 Readiness (model)	test_ai (2, ■ scaffold)	—	—	—	—
F4 Calorie	test_calories (14)	via test_checkin_flow / test_dashboard_flow	— (value surfaces with the model)	—	—
F7 Action plan (web surfaces list)	—	test_web_ai (2) · test_dashboard_flow (5)	test_e2e leg	—	—
F8 Dashboard	—	test_dashboard_flow (5)	test_e2e leg	—	test_dashboard (2)
F9 History	—	test_history_flow (3)	test_e2e leg	—	test_history (2)
Online Forum (CRUD/UI)	test_forum_validation (16)	test_forum_flow (11)	test_e2e leg	test_load (post flood → 429, in the compose-e2e gate)	test_forum (9)
Online Forum (received-engagement metric)	test_db_engagement (6)	test_engagement (7)	—	—	(auth-gated + counts-only, in test_engagement)
Media × engagement × caps (cross-feature journey)	—	test_elad_lane_journey (4, in-process)	test_elad_lane_live (real HTTP, incl. the 10 MB cap at the WSGI layer; compose-e2e)	—	(journey ends with the full auth-g sweep)
Data layer (db.py + Mongo)	test_db (36) · test_backup_script (2)	test_db_mongo (6, real Mongo)	—	—	(injection-safe queries, in test_profile / test_forum)
Frontend (SPA / CSRF / a11y)	—	test_frontend (12)	test_e2e leg	—	test_csrf (6) · test_web_hardening (4)
Observability (logging)	test_logging_config (19)	—	—	—	(access-log path escaping, in test_logging_config)
App config / debug flag / secret	test_config (8)	—	—	—	test_web_hardening (4)
Contract / skeleton / smoke	test_scaffold (4)	test_skeleton_contract (8) · test_web_smoke (1)	test_e2e (1)	—	—
Whole-system journey	—	—	test_e2e (register→profile→checkin→dashboard→history→forum→logout)	—	—
Load / abuse	—	—	—	test_load (4, live-stack burst: /health, /login flood, forum flood, /ready) · locustfile.py (ramped, on-demand CI job)	—

Totals by type (full suite): Unit 307 · Integration 339 · System 15 · Stress 12 · Security 91 → **764 tests**.

Pass / skip: locally **282 pass, 10 skip** in ~6 s. The 10 skips are *environment-gated, not broken* — they run the moment their dependency is present:

- test_db_mongo (6) — the real-Mongo integration suite; skips without TEST_MONGO_URI , and runs in CI against a mongo:7 service on every PR.
- test_e2e (1) — the full-stack system journey; runs when E2E_BASE_URL points at a live stack.
- test_ai (2) and test_load (1) — TDD scaffolds for the owners still building those planes (the AI model and the locust load scenario); they fail loudly (NotImplementedError) once un-skipped, so they can't rot into false green.

With the live stack up (`E2E_BASE_URL` + `TEST_MONGO_URI` set), the real-Mongo IT and the system e2e execute against the running containers too – **289 pass / 3 skip** (only the unwritten AI-model and locust scaffolds stay skipped).

No test is commented-out or `assert True` (course rule: a broken test is deleted or fixed, never muted).

4. Test strategy – the five course types

All five live under `tests/{Unit,Integration,System,Stress,Security}_Tests/`, run on any machine (env vars, no local paths), and are exercised by CI on every PR. A run that collects **0 tests fails** the gate.

- Unit (180)** – pure functions and single components in isolation: input validators (auth / profile / check-in / forum), the calorie formula, the `db.py` CRUD and Mongo-internals logic (against an in-memory fake), the logging configuration, and app config. Heavily parametrized on boundary and adversarial inputs (e.g. profile validation rejects injection objects, bools-as-numbers, and out-of-range values across every field).
- Integration (72)** – components wired together through the Flask test client: the auth flow, profile round-trip, check-in → `ai / predict` → persist, dashboard, history, and the full forum lifecycle. Two boundary suites are notable: `test_web_ai` stubs the `web` → `ai` HTTP seam with a contract-shaped response and asserts the dashboard surfaces it (and degrades to `ai_status: unavailable` when `ai` returns nothing); `test_db_mongo` runs the data layer against a **real** MongoDB.
- System (1, + live legs)** – `test_e2e` drives a complete user journey (register → profile → check-in → dashboard → history → forum vote/comment → logout → 401) over HTTP against a live 3-container stack. It is the end-to-end proof that the planes integrate.
- Stress (1 scaffold)** – `test_load` is the placeholder for a locust scenario (flood posts/votes → expect 429, not a crash), owned by the deploy/scale plane; runs on demand, not per-commit.
- Security (38)** – password hashing and session/auth-gating (`test_auth`), CSRF double-submit (`test_csrf`), NoSQL-injection-safe queries (`test_profile` , `test_forum`), ownership enforcement on forum edit/delete (403), auth-gating of dashboard/history, and response-hardening (`test_web_hardening`).

How to run

```
pytest -q                                     # the whole suite (real-Mongo + e2e skip)
TEST_MONGO_URI=mongodb://localhost:27017/worksmarter_test \
  pytest tests/Integration_Tests/test_db_mongo.py      # real-Mongo integration
E2E_BASE_URL=http://localhost:8000 pytest tests/System_Tests # full-stack system journey
```

CI & verification depth. CI is ruff → bandit → pytest, plus a `mongo:7` service so the real-Mongo suite executes on every PR; `main` is branch-protected and PR-only. Beyond CI, the web/data logic added this cycle was **mutation-tested** (deliberate one-line breakages must be caught by ≥1 test) and reviewed by independent adversarial passes – the concurrency, logging, fault-tolerance, and Mongo-provisioning paths were hardened against findings from those passes.

5. Risk assessment

Fault tolerance is a graded requirement: a dependency failing must **degrade, not crash** the app. Each row below names a way this system can fail, what the user loses when it does, the mitigation, and – the column that matters – the **test that would go red if the mitigation were removed**. A mitigation with no test behind it is an intention, and is marked as such.

A note on how these were chosen. The rows are not a checklist copied from a lecture; each one is either a failure we induced (stopping containers, flooding the queue, swapping the executor, breaking an invariant to watch a guard fire) or a failure mode the architecture makes reachable and we chose to bound in advance. Where a mitigation is deliberately *not* built, the reasoning is given rather than the row being quietly dropped.

5.1 Dependency failure (the mandatory fault-isolation requirement)

Risk	Impact	Mitigation	Status
ai container down / slow	No readiness or recommendations	<code>ai_client</code> wraps the call in try/except + timeout; the dashboard returns <code>ai_status: unavailable</code> and still renders; <code>web</code> boots even if <code>ai</code> is down (compose <code>service_started</code> , not <code>service_healthy</code>)	✅ built & tested (<code>test_ai_client</code> , <code>test_web_ai</code> degrade case; <code>test_fault_isolation.py</code> stops the real container)
db (Mongo) down	History can't be read/written	Store calls are try/except-guarded; forum/list routes return 503, not 500; <code>/health</code> stays 200 (Mongo-independent) so Docker's healthcheck does not restart a healthy <code>web</code> ; <code>/ready</code> degrades to 503	✅ built & tested (<code>test_fault_isolation.py</code> stops the real <code>db</code> container and re-probes over HTTP)
A restart storm under load	Cascading outage	<code>/health</code> is liveness only – un-rate-limited and Mongo-independent. A 429/5xx there would make Docker + UptimeRobot restart containers <i>because they are busy</i>	✅ built & tested (<code>locustfile.py</code> asserts <code>/health</code> is always 200 under load)
Wearable / smartwatch data missing	Advanced metrics absent	Fall back to manually entered values	✅ by design (no wearable dependency in the core path)

5.2 The AI job queue (GUIDELINES §2) – new surface, new failure modes

Putting a queue and a process pool in front of the model buys parallelism and introduces new ways to fail. Each is bounded on purpose. (The last two rows came out of the pre-submission adversarial review – both were real gaps in this table's earlier claims, and both are now guarded by tests that were mutation-checked by disabling the fix.)

Risk	Impact	Mitigation	Status
Queue saturation under a flood	Unbounded memory growth, and a backlog so deep every caller has already timed out before its job is scored	The backlog is bounded (<code>AI_QUEUE_MAX_PENDING</code>). Past it <code>submit()</code> raises <code>QueueFull</code> and <code>/predict</code> sheds with 503 , which <code>web</code> already treats as "ai unavailable" and degrades. Memory is the lesser reason: a queue deeper than the callers' patience is pure waste — work is done for clients that left. Shedding early keeps the p95 honest, and a bigger VM only moves that cliff, it does not remove it	✅ built & tested (<code>test_queue_backpressure.py</code> : 4x the bound arrives, the excess is shed stays 200)
Job-store memory leak	Slow-fuse OOM: one entry per <code>/predict</code> , forever	Finished jobs expire (<code>AI_JOB_TTL_SECONDS</code>) and the store is hard-capped (<code>AI_MAX_JOBS</code>). TTL alone is insufficient — a burst inside one TTL window is still unbounded — so both exist	✅ built & tested (<code>test_jobqueue.py</code> reaping + cap; <code>test_queue_backpressure.py</code>)
The model raises	Queue wedges permanently at "full"; every later request 503s	The failure is captured per-job and the depth slot is released on the error path too, so capacity returns the moment the model does. A worker crash cannot kill the callback thread	✅ built & tested (<code>test_a_flood_of_failing_jobs_does_not_wedge_the_queue</code> , <code>test_one_failing_job_does_not_poison_the_next</code>)
The model hangs	A unicorn thread is pinned forever; <code>ai</code> stops serving	<code>/predict</code> waits with a bounded timeout (<code>AI_PREDICT_TIMEOUT_SECONDS</code>) and returns 504 . The job keeps running — a timeout is the caller giving up, not a cancellation — and the container stays healthy	✅ built & tested (<code>test_predict_returns_504_when_the_model_outruns_the_time</code> verified against the live container)
Backpressure over-admits	The bound is fiction; the queue exceeds <code>max_pending</code>	The depth check and the slot reservation happen under one lock, and <code>result()</code> waits on a per-job <code>settled</code> event so a caller can never observe a finished job with a stale <code>pending</code> count	✅ built & tested (<code>test_the_in_flight_depth_never_exceeds_the_bound_under_concurrent_s</code> , <code>test_stats_are_settled_by_the_time_result_returns</code> — this was a real bug, for 3 flake)
A worker process dies (OOM, segfault)	A <code>ProcessPoolExecutor</code> with a dead worker is <i>permanently</i> broken: every later <code>submit()</code> raises <code>BrokenProcessPool</code> , so one crash was a 500 on every <code>/predict</code> forever — silently, since <code>/health</code> never touches the queue	The queue self-heals : a <code>BrokenProcessPool</code> (on submit or resolving an in-flight job) replaces the pool, generation-guarded so N concurrent detections rebuild once. The submit is retried once on the fresh pool; the caller whose job died gets a retryable 503 , and the very next request is served normally. Rebuilds are counted in <code>/queue/stats</code> (<code>pool_rebuilds</code>)	✅ built & tested (<code>test_jobqueue.py</code> broken-pool section; <code>test_a_dead_worker_process_costs_one_503_not_a_500_forever</code>)
A worker hangs (vs. dies)	Its future never completes, so its depth slot is never returned; <code>max_pending</code> hangs = permanent 503 with no code path back	A hard wall-clock reaper (<code>AI_JOB_HARD_TIMEOUT_SECONDS</code> , default 120 s — pinned by a guard test to exceed the <code>/predict</code> deadline) abandons a job with no result: the slot is released, waiters get a timeout, and a late completion is detected and never double-releases. If every pool worker is presumed hung, the pool itself is replaced. Limit honestly stated: Python cannot kill a truly hung worker process from the parent — the stuck process is leaked until the container restarts	✅ built & tested (<code>test_jobqueue.py</code> hung-workers section, incl. the double-release race)

5.3 Scale and deployment

Risk	Impact	Mitigation	Status
Load beyond one ai worker	Requests serialize behind the model	Two multiplying axes: the in-container process pool (<code>AI_QUEUE_WORKERS</code>) and replicas (<code>--scale ai=N</code>). Measured: 2.86× and 1.60× respectively	✅ built & measured (<code>SCALING_REPORT.md</code> ; <code>test_pool_scaling.py</code>)
Someone "simplifies" the process pool to threads	Silent total loss of parallelism (and the +5): every mocked test still passes	The pool type is asserted, and a real CPU-bound test measures it. A thread pool scores 0.96× on that test — the GIL — and fails the >1.5× bar	✅ built & tested (<code>test_pool_scaling.py</code> , <code>test_ai_queue_contract.py</code>)
GET <code>/jobs/<id></code> under <code>--scale ai=N</code>	404s $\sim(N-1)/N$ of the time: the job store is per-container and the GET round-robins to a replica that never saw the job	<code>web</code> calls only <code>/predict</code> , which is replica-safe (one request, one response, nothing read back). Documented in three places and guard-tested	✅ bounded by design — not made replica-safe on purpose: an external store (Redis) would add a 4th container, a new failure mode and a new dependency to serve an endpoint nothing calls (<code>test_scale_contract.py</code>)
The CPU-burning benchmark ships as the production model	Every prediction slow and meaningless	<code>inference:predict_one</code> is the default target; a guard asserts no shipped compose file selects <code>bench:cpu_burn</code>	✅ built & tested (<code>test_scale_contract.py</code>)
A scaled ai gets published to the host	Port collision on replica 2, and an unauthenticated <code>/jobs</code> reachable from outside	Only <code>web</code> publishes a port, in dev, test, prod and the scale override. <code>ai</code> has no auth because nothing but <code>web</code> can reach it — that assumption is load-bearing, so it is asserted	✅ built & tested (<code>test_deploy_contract.py</code> , <code>test_ai_queue_contract.py</code> , <code>test_scale_contract.py</code>)
A bad deploy takes the public site down	Public app down	CI gates the image build; the build gates the deploy; post-deploy <code>/ready</code> is probed over HTTPS and a failure auto-rolls back to the previous image while still failing the run. <code>restart: unless-stopped</code> survives a VM reboot	✅ built & tested (<code>test_deploy_contract.py</code> locks the pipeline shape; live at <code>app.worksmarternotharder.dev</code>)
Single Azure VM (no redundancy)	Total outage if the VM dies	Accepted. The course supplies one VM; the containers are stateless, so a Swarm overlay or a second host is a configuration change, not a rewrite. UptimeRobot alerts on the outage rather than preventing it	🟡 accepted risk, mitigation documented (<code>DESIGN.md</code> §L7)
Two gunicorn workers on ai	GET <code>/jobs/<id></code> 404s $\sim$ half the time — each worker owns a separate in-memory job store, and the read lands on whichever one the OS picked	<code>ai</code> runs exactly one worker with threads; parallelism comes from the pool. This is architectural , not a memory budget — the 32 GiB VM would host a second worker comfortably, and the job store would still split in two	✅ built & tested (<code>test_ai_runs_exactly_one_gunicorn_worker</code>)
ai's gunicorn worker outlives its own predict deadline	A <code>/predict</code> that runs to <code>AI_PREDICT_TIMEOUT_SECONDS</code> races a worker SIGKILL: <code>web</code> sees a dropped connection, the in-flight job dies with the process, and the clean 504 degrade is unreachable	gunicorn's <code>--timeout</code> (60s) is pinned above the queue deadline (30s) in both the image and prod. gunicorn's own default is 30 — equal to the deadline — so leaving it unset was the bug	✅ built & tested (<code>test_ai_gunicorn_timeout_exceeds_the_queue_timeout</code> ; guard is mutation-tested against absent and too-low values)

5.4 Abuse, security and data

Risk	Impact	Mitigation	Status
Spam / request floods (login, register, forum)	Resource exhaustion, bulk signup, password guessing	<code>flask-limiter</code> on the public write routes; messaging carries its own 20/min anti-spam cap. Under load a 429 is the defence engaging , not a failure — only a 5xx is	✅ built & tested (<code>test_rate_limit.py</code> ; <code>locustfile.py</code> treats 429 as a pass and 5xx as a failure)
Huge-file upload (posts, comments, DMs)	Disk exhaustion, memory blowup	Per-upload size caps + type checks; <code>MAX_CONTENT_LENGTH</code> rejects an oversized body before Flask parses it	✅ built & tested (<code>test_media_limits.py</code>)
Hostile payload reaching a worker process	Wasted worker, potential injection	<code>/predict</code> validates the body shape and caps the feature count before <code>submit()</code> , so a rejected request costs a 400, not a pickled round-trip into a child process	✅ built & tested (<code>test_ai_queue.py</code> : injection-shaped keys, feature bombs, oversized bodies)
Job-id enumeration	Reading another user's readiness score (<code>ai</code> has no auth by design)	Job ids are <code>uuid4</code> hex, unguessable and non-sequential; an unknown id is a flat 404 with no detail; a model traceback never reaches the caller	✅ built & tested (<code>test_ai_queue.py</code>)
NoSQL injection	Query manipulation, data leak	All inputs type-validated at the route layer before any query; <code>db.py</code> uses parameterized filters, never raw user objects	✅ built & tested (<code>test_profile</code> , <code>test_forum</code> , validation unit suites)
Credential theft / weak auth	Account takeover	<code>werkzeug</code> password hashing (never plaintext); protected endpoints auth-gated; CSRF double-submit on state-changing requests	✅ built & tested (<code>test_auth</code> , <code>test_csrf</code>)
Forum abuse — edit/delete others' posts	Integrity/trust	Ownership check → 403 on non-owner edit/delete	✅ built & tested (<code>test_forum</code>)
Concurrent forum votes / duplicate writes	Lost updates, double-counts	Optimistic-concurrency (compare-and-set) vote loop with bounded retries; atomic upsert dedupe; TOCTOU-safe edit/delete via matched-count	✅ built & tested (<code>test_db</code> , real-Mongo <code>test_db_mongo</code>)
Malformed documents reaching the DB	Corrupt reads, crashes	<code>\$jsonSchema</code> validators on all collections (defence-in-depth behind route validation) + unique/perf indexes	✅ built (<code>ensure_schema</code> / <code>ensure_indexes</code>), exercised by the real-Mongo IT
Data loss (Mongo volume)	Irrecoverable history	<code>db/backup.sh</code> (gzip dump + retention prune); named volume; fails fast if <code>MONGO_URI</code> unset	✅ built & tested (<code>test_backup_script</code>)
Log files growing unbounded	Disk exhaustion	<code>RotatingFileHandler</code> (size-capped, per-worker files); logging fully disableable via env flag	✅ built & tested (<code>test_logging_config</code>)
Secrets committed to the repo	Credential leak	<code>.env</code> git-ignored; <code>.env.example</code> only; a scaffold test asserts no <code>.env</code> is tracked; the AI model is local → no API keys shipped	✅ built & tested (<code>test_scaffold</code>)

5.5 What we deliberately did not mitigate

Naming these is part of the assessment; an unlisted risk is an unnoticed one.

- `/jobs` is **not replica-safe**. Fixing it means Redis or Mongo-backed job state — a fourth container, a new dependency and a new failure mode, serving an endpoint the app does not call. The synchronous `/predict` path needs none of it. This was never a memory decision, so the larger VM does not change it.
- One VM, no failover**. One machine is what the course provides — and it is the single point of failure that actually matters. It was resized mid-project to a **Standard E4ads v5 (4 vCPU / 32 GiB)**, which bought throughput, not availability. Recovery is `restart: unless-stopped` plus an UptimeRobot alert, not redundancy.
- `web` is **not replicated**. Sessions are cookie-signed, so it would scale horizontally, and since the VM resize there is RAM to spare for it. We still don't: `web` is I/O-bound (it waits on Mongo and `ai`), so a wider **thread** pool inside one gthread worker serves the same concurrency more cheaply than a second process, and replicas of `web` on the one VM add no availability — the VM is still the thing that dies.
- Tail latency across `ai` replicas**. Docker's DNS round-robin balances connections, not work, so p95 barely improves with replicas. A real load balancer is not warranted for one VM.

5.6 How the mitigations are kept honest

Two habits, both visible in the test suite:

- Guard tests over prose**. The invariants that a well-meaning future change can silently break — `ai` stays internal, the queue stays bounded, the pool stays processes, `/predict` still goes through the queue, `predict_one` keeps its shape — are asserted in `test_deploy_contract.py`, `test_ai_queue_contract.py` and `test_scale_contract.py`, so the breakage surfaces as a red PR rather than as a dead container after a merge.
- Mutation testing**. Every guard above was verified by *breaking the invariant on purpose* and confirming the guard went red: bypass the queue, swap the pool to threads, unbound the backlog, rename `predict_one`, restore a second gunicorn worker, leak the depth counter, delete the reaping, publish an `ai` port. A guard that cannot fail is decoration.

The `ai` / `db` fault-isolation requirement, the queue's failure modes, and the web tier's security surface are covered by tests that run on every PR. The remaining open rows belong to the AI model (Shiri) and are tracked in [ROADMAP.md](#).

6. Division of labor (current split)

Roles are **containers/aspects**, not a rigid feature list. The only fixed shared things are the contracts between planes (§1). Full detail in [COLLABORATORS.md](#) and the `PERSON{1,2,3}.md` files.

Person	Plane	Owns
Shiri (shiriHaboob)	AI brain	Random Forest model, real /predict , recommendation engine, the dataset, forum cold-seed content; AI unit tests
Lior (LiorBenSidi)	Web app + data + observability + CI/CD	Flask backend (API · auth/sessions · validation · ai+db orchestration) + frontend (incl. the Forum real-time layer — P2P DM · live SSE notifications · vote-notifications · comment-votes) + the whole data layer (db.py · CRUD, indexes, \$jsonSchema validators, auth config, backups, seed) + Week-9 logging + the web / ai container build & compose + the CI gate + the CI/CD deploy pipeline (build→GHCR→SSH-deploy→Caddy HTTPS, docker-compose.prod.yml); the web/data integration/system/security tests
Elad (EladNa1)	Live deployment + scale + forum media	the live Azure deploy (VM secrets, GHCR-packages-public, UptimeRobot, the deploy demo — pipeline code is Lior's), the test-runner service, the Forum media/attachments (+ file-size caps), rate-limiting (flask-limiter), stress + cross-container tests

7. How to run / reproduce

```
cp .env.example .env
docker compose up --build          # 3 containers; only web is published
# → http://localhost:8000/health   → then register, set a profile, check in, see the dashboard
pytest -q                          # 282 pass / 10 env-gated skip
```

CI reproduces the gate on every PR (ruff → bandit → pytest + a mongo:7 service). main is branch-protected and merges only on green CI.

8. Open items (tracked; not blockers for the built planes)

- **F6 + F7 merge** — awaiting Noam's confirmation; the submitted 9-feature scope is the graded contract until he replies. Fallback (trend-analysis-over-history) is defined in ROADMAP.md .
- **Exact readiness categories + final feature set** — finalized by the AI plane during data exploration (the /predict contract already pins the shapes: features in, {state, proba, recommendations, calories} out).
- **Deploy ordering** (Azure right after the MVP vs after the 80) and the **forum real-time transport** (SSE vs Flask-SocketIO) — deploy/real-time plane decisions.

This report is regenerated against the real suite as the build advances; the matrix in §3 always reflects tests/ at the stated snapshot date.